VITA-FLOW

GitHub: https://github.com/Karan24-8/VITA-FLOW | Live: https://vita-flow-pi.vercel.app

1. Member Contributions

ID	Name	Contribution
2024A7RM0236G	Yashashwi Vundavalli	Frontend CSS styling and API connections
2024A7PS0533G	Ujjwal Gupta	Exercise module DB implementation and AI module
2024A7PS0549G	Ansh Kumar	Consultant module DB implementation and AI module
2024A7PS0571G	Abhinav Sharma	Food module DB implementation and role-based access
2024A7PS0855G	Krishav Goyal	Frontend HTML & JS and API connections
2024A7PS0496G	Karan Pote	Complete backend, API routes, DB connection, AI integration

2. Technology Stack

Layer	Technology	Description
Frontend	HTML5, CSS3, Vanilla JS	Responsive pages, modular JS, dynamic DOM updates via fetch()
Backend	Node.js + Express.js	REST API on Render; JWT auth middleware; modular route controllers
Database	PostgreSQL via Supabase	Normalized relational schema (3NF); PL/pgSQL stored procedures
AI	Claude / OpenAI API	Integrated for diet plans, workout suggestions, consult summaries
Deployment	Vercel (FE) + Render (BE)	Frontend on Vercel CDN; backend as always-on Render service

3. User Roles and Privileges

The platform implements Role-Based Access Control (RBAC) with three user types:

user

- View and update own profile.
- Generate own AI diet and workout plans.
- View the list of consultants.

consultant

- View and update own profile.
- View all registered users.
- Update any user's diet and workout plan.
- View the list of consultants.
- Cannot generate own diet or workout plan.

DBA

- Everything a consultant can do.
- View all users via the user route.
- Cannot generate own diet or workout plan.

4. Modules Developed

Food & Diet Module

Developed by: Abhinav Sharma (DB) | Yashashwi & Krishav (Frontend)

- Search and log daily food intake from a nutritional database (calories, protein, carbs, fats, fiber).
- Set daily calorie targets; module computes running totals and displays progress.
- AI generates personalized meal plans based on user goals and dietary history.

Exercise Module

Developed by: Ujjwal Gupta (DB + AI) | Krishav & Yashashwi (Frontend)

- Comprehensive exercise library categorized by muscle group, equipment, and difficulty.
- Users create workout plans, log sessions (sets, reps, duration), and track weekly progress.
- AI generates personalized workout routines; DB supports streak and volume calculations.

Consultant Module

Developed by: Ansh Kumar (DB + AI) | Krishav & Yashashwi (Frontend)

- Browse consultants (dieticians, trainers), view availability, and book appointments.
- Manages appointment status: pending → confirmed → completed; stores consultation notes.
- AI pre-generates a user health summary to assist the consultant before the session.

Authentication & Role Management

Developed by: Karan Pote (Backend) | Abhinav Sharma (RBAC)

- Secure registration and login using JWT tokens; passwords hashed with bcrypt.
- Three roles — user, consultant, DBA — enforced via Express middleware on every route.
- Profile management endpoints update health parameters used across all modules.

5. Database Schema

PostgreSQL database hosted on Supabase. Schema follows Third Normal Form (3NF). Complex aggregations are handled via PL/pgSQL stored procedures.

Table	Key Columns	Linked To
users	user_id (PK), name, email, password_hash, role, age, weight, height, goal	All user-specific tables
food_items	food_id (PK), name, calories, protein, carbs, fat, fiber, category	user_food_logs
user_food_logs	log_id (PK), user_id (FK), food_id (FK), quantity, log_date, meal_type	users, food_items
exercises	exercise_id (PK), name, category, muscle_group, difficulty, equipment	workout_sessions
workout_sessions	session_id (PK), user_id (FK), exercise_id (FK), sets, reps, duration, date	users, exercises
consultants	consultant_id (PK), name, specialization, bio, availability_slots	appointments
appointments	appt_id (PK), user_id (FK), consultant_id (FK), slot, status, notes	users, consultants
ai_logs	ai_log_id (PK), user_id (FK), module, prompt_summary, timestamp	users

6. Frontend Overview

Built with pure HTML5, CSS3, and Vanilla JavaScript — no frameworks — for fast load times. Each module has a dedicated HTML page; shared utilities are abstracted into reusable JS files. All pages communicate with the backend via the Fetch API using JWT tokens. A central dashboard aggregates data across all modules. CSS custom properties maintain a consistent teal-based health theme throughout.

7. Backend & API Overview

Node.js + Express.js REST API with a modular route structure — each module has its own router. Middleware chains handle JWT verification, role authorization, and error formatting before requests reach controllers. Key route groups:

Route Group	Endpoints	Access
Auth	/api/auth/register | /api/auth/login	Public
Food	/api/food | /api/food/log | /api/food/summary	user
Exercise	/api/exercise | /api/exercise/session | /api/exercise/history	user
Consultant	/api/consultants | /api/appointments	user, consultant, DBA
AI	/api/ai/diet | /api/ai/workout | /api/ai/consult	user
Admin	/api/admin/users | /api/admin/stats	consultant, DBA